PROJECT CHARTER & SPECIFICATION

Semivra Libellus POS

1. Project Overview

A full-stack, offline-capable Point-of-Sale and back-office ERP for F&B businesses (cafs/restaurants), built first for the "Kasa Lokal" brand and architected for white-label resale.

- **Project Name:** Semivra Libellus POS (codename: Negotium / Libellus)
- **Project Type:** Full-stack web app (PWA) + REST API + real-time backend
- **Target Start Date:** 2026-04 (initial build)
- **Target Completion Date:** 2026-06 (V1.0 pilot-ready)
- **Current Status:** In Progress - hardening complete on branch `security/hardening-p1-p3` (PR #1); awaiting staging verification + merge

Executive Summary

- **The Problem:** Small/independent PH F&B operators are stuck between toy POS apps (no real accounting/ inventory) and expensive, US-tax-centric, online-only platforms (Square/Toast) that charge per-transaction fees and don't fit Non-VAT/BIR realities. Reconciliation, COGS, and tax reporting are manual and error-prone.
- **The Solution:** One tablet-first app that unifies POS, recipe-level inventory with FEFO expiry, true double-entry accounting, shift/cash control, and QR self-ordering - Non-VAT/BIR-aware out of the box, offline-first, and white-labelable.
- **Business Impact:** Eliminates manual bookkeeping (every sale/void/expense auto-posts a balanced journal entry), gives real-time COGS and waste visibility, prevents oversell via atomic stock deduction, and produces export-ready P&L / Balance Sheet - turning a caf's register into a closed-books ERP.

2. Stakeholders & Core Team

Role	Name	Responsibility / Touchpoint
Project Sponsor / Owner	s3mivra (owner)	Decision-maker, scope sign-off, business model
Project Manager / Lead	s3mivra	Execution, priorities, deployment
Technical Lead / Architect	s3mivra (+ AI pair)	System design, code quality, stack decisions
Core Contributors	s3mivra	Full-stack development
Key Stakeholders	Kasa Lokal staff/cashiers; accountant (BIR review); future white-label clients	End users; compliance sign-off

3. Scope & Requirements

In-Scope (Must-Haves for Launch)

- **POS register:** search, modifiers/sizes/add-ons, PHP /% discounts, multi payment methods, change calc, park/ recall, voids, refunds, complimentary.
- **Fulfilment modes:** Dine-In, Takeout, Pickup, Manual Delivery, Grab, Foodpanda (delivery channels booked to A/R).

- **Inventory:** recipe-based deduction, FEFO multi-batch expiry, restock/procurement/spoilage, Excel/CSV import + stock-take, low-stock alerts; kg/L/pcs display with base-unit storage.
- **Accounting:** double-entry GL, P&L, Balance Sheet, A/R, A/P, expenses, revolving funds, Non-VAT percentage tax; balanced-entry assertions; all money/stock moves inside DB transactions.
- **Shifts & cash:** starting float, EOD reconciliation + variance journal, bank deposit, shift history, X-Reading; mandatory staff clock-in.
- **Auth & security:** dual-token auth (15m access + httpOnly refresh) with server-side revocation, RBAC (superadmin/staff), Helmet, Zod validation, rate limiting.
- **QR self-ordering:** time-boxed single-use sessions.
- **Platform:** real-time sync (Socket.io), offline order queue (PWA), tablet-optimized UI.

Out-of-Scope (Future Phases)

- Native mobile apps (PWA only for now)
- Multi-tenant data isolation (currently single-tenant per deployment)
- Loyalty / CRM / marketing module
- AI-driven demand forecasting / predictive analytics
- Automated per-client provisioning for white-label resale

---

4. Technical Architecture & Stack

System Architecture

- **Deployment Paradigm:** Monolithic API + SPA (single-tenant per deployment)
- **Hosting/Cloud Infrastructure:** Backend on **Railway**, Frontend on **Vercel**; Docker + docker-compose + nginx provided for self-host

Technology Stack

- **Frontend:** React 18 Vite 8 (rolldown) Tailwind CSS v3 lucide-react socket.io-client PWA (service worker, offline queue)
- **Backend:** Node.js Express 4 Socket.io JWT (dual-token) pino logging optional Sentry
- **Database:** MongoDB (Mongoose 9) - transactions for order/void/refund/count/import
- **DevOps/CI-CD:** GitHub Actions (server lint/test/build + client build) Docker Playwright (E2E scaffold) Vitest (unit)

---

5. Milestone & Timeline Tracking

```
[Phase 1: Build] > [Phase 2: Hardening] > [Phase 3: Verify/Pilot] > [Phase 4: Launch/Sell]
       DONE                 DONE                   IN PROGRESS                PENDING
```

- **Milestone 1: Core build** - COMPLETE. POS, inventory, accounting, shifts, QR, PWA.
- **Milestone 2: Security & QA hardening** - COMPLETE (branch `security/hardening-p1-p3`, 21 commits). Dual-token auth, Helmet/Zod/rate-limit, bug fixes, docs, CI green, 81 unit tests.
- **Milestone 3: Verification & Pilot** - IN PROGRESS. Browser verification of auth -> merge -> run Kasa Lokal live 2-4 weeks -> accountant BIR review.
  - *Deliverables:* `GO_LIVE.md` walkthrough passed, production env set, Sentry on, books reviewed.
- **Milestone 4: Production V1.0 / sellable** - PENDING.
  - *Deliverables:* live deployment, monitoring active, per-instance vs multi-tenant decision, legal/pricing/support for resale.

---

6. Risk Assessment & Mitigation

- **Risk 1: Breaking auth change locks users out on deploy** - *Impact: High* - *Mitigation:* `GO_LIVE.md` staging walkthrough + correct `ALLOWED_ORIGINS`/HTTPS; non-rotating refresh removes the reload-logout race; additive DB changes mean code rollback needs no DB rollback.
- **Risk 2: BIR/Non-VAT accounting incorrectness** - *Impact: High* - *Mitigation:* balanced-entry assertions in code; **mandatory accountant review** of the first month before selling.
- **Risk 3: Single-tenant limits resale** - *Impact: Medium* - *Mitigation:* per-instance deployment for the first few clients; multi-tenant as a dedicated future project.
- **Risk 4: Delivery-partner / payment channel reconciliation** - *Impact: Medium* - *Mitigation:* non-cash channels book to A/R until explicitly settled with chosen deposit account.
- **Risk 5: Tablet performance (large admin bundle)** - *Impact: Low* - *Mitigation:* routes already lazy-loaded; heavy libs (xlsx/html2canvas) code-split; targets 12GB-RAM tablets.

---

7. Definition of Done (DoD)

- [x] **Code Quality:** server `node --check` clean; client `vite build` clean; CI green.
- [~] **Testing:** 81 unit tests (ledger/units/expiry/chart-of-accounts); Playwright E2E scaffolded (not yet run); broader integration coverage pending.
- [x] **Security:** no hardcoded secrets; env fail-fast; RBAC + ownership checks on endpoints; 0 npm vulnerabilities.
- [x] **Documentation:** `MANUAL.md` (+PDF), `GO_LIVE.md`, `AUDIT_OVERVIEW.md`, `.env.example`, this charter.
- [ ] **Verified live:** auth flow proven in a browser; one real shift run end-to-end.

---

8. Appendix & Resources

- **Code Repository:** https://github.com/s3mivra/local-based-menu
- **Pull Request:** PR #1 - `security/hardening-p1-p3` -> `main`
- **Backend (prod):** Railway **Frontend (prod):** Vercel
- **User Manual:** `MANUAL.md` / `Semivra_Libellus_Manual.pdf`
- **Deploy Runbook:** `GO_LIVE.md`
- **Audit & Competitive Analysis:** `AUDIT_OVERVIEW.md`
- **Session Report:** `SESSION_REPORT.md`
- **Environment Variables:** see `server/.env.example` and `GO_LIVE.md` 1